



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 数据结构与算法导论

实验名称: 实验 2: ADT 栈与队列的实现

专 业 班 级 通信工程 三班

学 生 学 号 _____

学 生 姓 名 _____

实 验 时 间 2025 年 5 月 18 日

实验报告

【实验目的】

加深对抽象数据类型ADT栈和队列的理解；

【实验原理】

1. 栈

栈（stack）是只能在一个位置上进行插入和删除元素操作的表。（先入后出）该位置是表的末端，叫做栈的顶（top）。对栈的基本操作有 Push（压栈）和 Pop（出栈），前者相当于插入，后者则是删除最后插入的元素。

实现栈有两种方式，分别是指针链表实现与数组实现，这里我们以链表在栈顶进行插入和删除操作来实现压栈与弹栈操作。

2. 队列

队列（Queue）是一个表。它只能在首位删除元素，在末尾插入元素（先入先出）对栈的基本操作有 EnQueue（入队）和 DeQueue（出队），前者相当于插入，后者则是删除最后插入的元素。

实现队列有两种方式，分别是指针链表实现与数组实现，这里我们以链表在队头进行删除和插入操作来实现入队与出队操作。

【实验要求】

编写程序实现ADT栈的定义，及常用操作（数组或指针实现）：

- 1) 生成栈；
- 2) Push
- 3) Pop

编写程序实现ADT队列的定义，及常用操作：

- 1) 生成队列；
- 2) Enqueues入列；
- 3) Isempty判断是否队列为空

【实验源程序】

以实验 1 ADT 表函数，分别实现栈与队列：

1.ADT 头文件

```
#ifndef _ADT_H
#define _ADT_H

typedef struct Node {
    int data;
    struct Node* next;
} Node;

typedef struct {
    Node* head;
    int size;
} List;

void initList(List* list); //链表初始化
int isEmpty(List* list); //是否非空
int getElement(List* list, int i, int* element); //遍历链表至第 i 个元素
int insertAt(List* list, int i, int data); //插入操作
int deleteAt(List* list, int i); //删除操作
void deleteList(List* list); //删除链表

#endif
```

2.ADT 函数文件

```
#include <stdio.h>
#include <stdlib.h>

/* 链表节点结构体
```

```

    * 包含数据域和指向下一个节点的指针 */
typedef struct Node {
    int data;           // 节点存储的数据
    struct Node* next;  // 指向下一个节点的指针
} Node;

/* 链表结构体
 * 包含链表头指针和记录链表长度的 size */
typedef struct {
    Node* head; // 指向链表第一个节点的指针
    int size;    // 记录链表当前节点数量
} List;

/* 初始化链表
 * 将链表头指针置空, size 置 0
 * 参数: list-要初始化的链表指针 */
void initList(List* list) {
    list->head = NULL;
    list->size = 0;
}

/* 判断链表是否为空
 * 参数: list-链表指针
 * 返回值: 1 表示空表, 0 表示非空表 */
int isEmpty(List* list) {
    return list->size == 0;
}

/* 获取链表中第 i 个元素的值 (i 从 1 开始计数)
 * 参数: list-链表指针, i-元素位置, element-用于存储获取值的指针
 * 返回值: 1 成功获取, 0 获取失败 (i 越界) */
int getElement(List* list, int i, int* element) {
    // 检查位置有效性 (1 ≤ i ≤ size)
    if (i < 1 || i > list->size) {
        return 0;
    }

    // 从头节点开始遍历到第 i 个节点
    Node* current = list->head;
    for (int pos = 1; pos < i; pos++) {
        current = current->next;
    }

```

```

        *element = current->data; // 通过指针参数返回元素值
        return 1;
    }

/* 在链表第 i 个位置插入新节点 (i 从 1 开始, 允许插入到 size+1 位置)
 * 参数: list-链表指针, i-插入位置, data-插入的数据
 * 返回值: 1 插入成功, 0 插入失败 (i 越界或内存分配失败) */
int insertAt(List* list, int i, int data) {
    // 检查位置有效性 (1 ≤ i ≤ size+1)
    if (i < 1 || i > list->size + 1) {
        return 0;
    }

    // 创建新节点并分配内存
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (!newNode) {
        return 0; // 内存分配失败
    }
    newNode->data = data;

    if (i == 1) { // 插入到链表头部
        newNode->next = list->head;
        list->head = newNode;
    } else { // 插入到中间或尾部
        // 找到第 i-1 个节点作为前驱节点
        Node* prev = list->head;
        for (int pos = 1; pos < i - 1; pos++) {
            prev = prev->next;
        }
        // 将新节点插入到前驱节点之后
        newNode->next = prev->next;
        prev->next = newNode;
    }

    list->size++; // 更新链表长度
    return 1;
}

/* 删除链表中第 i 个节点 (i 从 1 开始)
 * 参数: list-链表指针, i-要删除的位置
 * 返回值: 1 删除成功, 0 删除失败 (i 越界) */
int deleteAt(List* list, int i) {
    // 检查位置有效性 (1 ≤ i ≤ size)

```

```

    if (i < 1 || i > list->size) {
        return 0;
    }

    Node* temp; // 用于保存要删除的节点指针
    if (i == 1) { // 删除头节点
        temp = list->head;
        list->head = temp->next; // 更新头指针
    } else {
        // 找到第 i-1 个节点作为前一个节点
        Node* prev = list->head;
        for (int pos = 1; pos < i - 1; pos++) {
            prev = prev->next;
        }
        temp = prev->next; // 保存要删除的节点
        prev->next = temp->next; // 前一个节点指向被删节点的下一个节点
    }

    free(temp); // 释放被删除节点的内存
    list->size--; // 更新链表长度
    return 1;
}

/* 销毁链表，释放所有节点内存
 * 参数: list-链表指针 */
void deleteList(List* list) {
    Node* current = list->head;
    // 遍历链表逐个释放节点内存
    while (current != NULL) {
        Node* temp = current; // 保存当前节点指针
        current = current->next; // 移动到下一个节点
        free(temp); // 释放当前节点内存
    }
    // 重置链表状态
    list->head = NULL;
    list->size = 0;
}

```

(1) 栈的实现

1. 定义栈头文件

```
#ifndef _STACK_H
#define _STACK_H

typedef struct
{
    List list;
}Stack;

void initStack(Stack* Stack);//初始化栈
int isEmpty_S(Stack* Stack);//判断栈是否为空
int Push(Stack* Stack,int data);//压栈
int Pop(Stack* Stack,int *element);//出栈
void del_Stack(Stack* Stack);//销毁栈

#endif
```

2. 栈操作的函数实现

```
#include <stdio.h>
#include <stdlib.h>
#include "ADT.h"
#include "Stack.h"

/* 初始化栈结构
 * 参数：Stack - 指向需要初始化的栈的指针
 * 功能：通过调用链表初始化函数来初始化栈的底层数据结构
 */
void initStack(Stack* Stack){
    initList(&Stack->list);
}

/* 判断栈是否为空
 * 参数：Stack - 指向要检查的栈的指针
 * 返回：1 表示栈空，0 表示非空
 */
int isEmpty_S(Stack* Stack){
    isEmpty_L(&Stack->list);
}
```

```

}

/* 元素压栈操作
 * 参数: Stack - 目标栈指针, data - 要压入的整型数据
 * 返回: 操作结果 (成功/失败)
 * 特点: 在链表头部插入实现后进先出特性
 *       插入位置 1 表示在链表第一个节点位置插入
 */
int Push(Stack* Stack,int data){
    insertAt(&Stack->list , 1 , data);
}

/* 元素弹栈操作
 * 参数: Stack - 目标栈指针, element - 用于存储弹出值的指针
 * 返回: 操作结果 (成功 1/失败 0)
 * 流程: 1. 检查栈空状态
 *       2. 获取链表第一个节点的数据
 *       3. 删除链表第一个节点
 */
int Pop(Stack* Stack,int *element){
    if (isEmpty_S(&Stack->list)) {
        return 0; // 栈空
    }
    getElement(&Stack->list, 1 , element);
    deleteAt(&Stack->list, 1);
}

/* 销毁栈结构
 * 参数: Stack - 要销毁的栈指针
 * 功能: 通过销毁底层链表释放栈占用的所有内存资源
 */
void del_Stack(Stack* Stack)
{
    destroyList(&Stack->list);
}

```

3.主函数应用

```

#include <stdio.h>
#include <stdio.h>
#include <stdlib.h>
#include "ADT.h"
#include "Stack.h"

```

```
int main(){
    Stack stack;
    initStack(&stack);

    Push(&stack,10);//压 10
    Push(&stack,5);//压 5
    Push(&stack,1);//压 1

    printf("栈是否为空: %s\n", isEmpty_S(&stack) ? "是" : "否"); // 输出否

    int elem;
    if (Pop(&stack, &elem)) {
        printf("第一个元素: %d\n", elem); // 输出 1
    }
    if (Pop(&stack, &elem)) {
        printf("第二个元素: %d\n", elem); // 输出 5
    }
    if (Pop(&stack, &elem)) {
        printf("第三个元素: %d\n", elem); // 输出 10
    }
    printf("栈是否为空: %s\n", isEmpty_S(&stack) ? "是" : "否"); // 输出是
    del_Stack(&stack);
    return 0;
}
```

(2) 队列的实现

1. 队列头文件

```
#ifndef _Queue_H
#define _Queue_H

#include <stdio.h>
#include <stdlib.h>
#include "ADT.h"

typedef struct{
    List list;
```

```

}Queue;

void initQue(Queue* Queue); //初始化队列
int isEmpty_Q(Queue* Queue); //判断队列是否为空
int enQue(Queue* Queue, int data); //入队
int deQue(Queue* Queue, int *element); //出队
void del_Queue(Queue* Queue); //销毁队列

#endif
    
```

2. 队列操作函数

```

#include <stdio.h>
#include <stdlib.h>
#include "ADT.h"
#include "Queue.h"

/*
 * 初始化队列
 * 参数 Queue: 指向要初始化的队列的指针
 * 该函数通过初始化内部链表来准备队列，必须在其他操作前调用
 */
void initQue(Queue* Queue){
    initList(&Queue->list); // 调用链表初始化函数
}

/*
 * 判断队列是否为空
 * 参数 Queue: 指向队列的指针
 * 返回值: 1 表示队列为空, 0 表示非空
 */
int isEmpty_Q(Queue* Queue){
    return isEmpty_L(&Queue->list); // 直接返回链表判空结果
}

/*
 * 将元素加入队列 (入队操作)
 * 参数 Queue: 指向队列的指针
 * 参数 data: 要入队的整型数据
 */
void enQue(Queue* Queue, int data){
    if(Queue->list.size == 0){ // 处理空队列情况
        insertAt(&Queue->list, 1, data); // 首元素插入位置 1
    }
}
    
```

```

        return;
    }
    insertAt(&Queue->list ,(Queue->list.size)+1, data); // 非空时插入当前尾端
    return ;
}

/*
 * 从队列移除元素（出队操作）
 * 参数 Queue: 指向队列的指针
 * 参数 element: 用于存储取出元素的指针
 */
void deQue(Queue* Queue,int *element){
    if (isEmpty_Q(Queue)) {
        printf("Error\n");
        return ; // 空队列直接返回失败
    }
    getElement(&Queue->list, 1 , element); // 获取队首元素
    deleteAt(&Queue->list, 1); // 删除队首元素
    return ;
}

/*
 * 销毁队列并释放内存
 * 参数 Queue: 指向要销毁的队列指针
 */
void del_Queue(Queue* Queue)
{
    destroyList(&Queue->list); // 调用链表销毁函数
}

```

3.主函数应用

```

#include <stdio.h>
#include <stdlib.h>
#include "ADT.h"
#include "Queue.h"

int main()
{
    Queue queue;
    initQue(&queue); // 初始化队列

    enQue(&queue,1); // 1

```

```

enQue(&queue,2);//1->2
enQue(&queue,3);//1->2->3

printf("队列是否为空: %s\n", isEmpty_Q(&queue) ? "是" : "否"); // 输出否

int elem;
deQue(&queue, &elem);
printf("第一个元素: %d\n", elem); // 输出 1

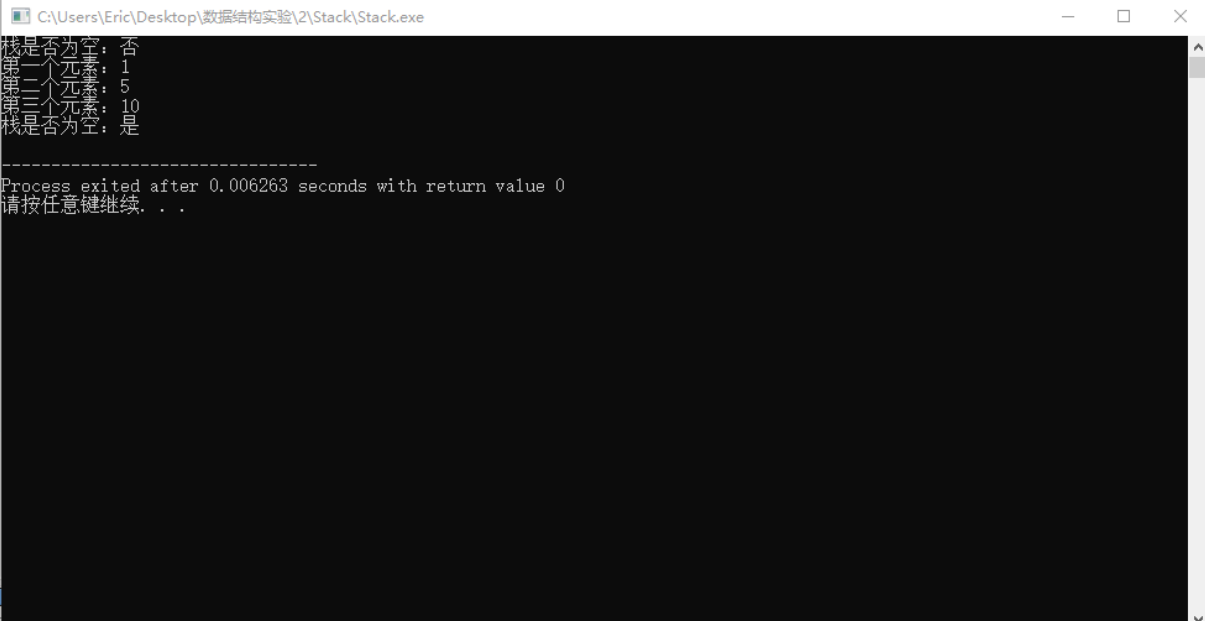
deQue(&queue, &elem);
printf("第二个元素: %d\n", elem); // 输出 2

deQue(&queue, &elem);
printf("第三个元素: %d\n", elem); // 输出 3

printf("队列是否为空: %s\n", isEmpty_Q(&queue) ? "是" : "否"); // 输出是
del_Queue(&queue);
return 0;
}
    
```

【实验结果】

1. 栈的实现



```

C:\Users\Eric\Desktop\数据结构实验\2\Stack\Stack.exe
栈是否为空: 否
第一个元素: 1
第二个元素: 5
第三个元素: 10
栈是否为空: 是

-----
Process exited after 0.006263 seconds with return value 0
请按任意键继续. . .
    
```

图 1 运行结果截图

由于我们先输入的是 10, 5, 1, 这里输出的就是 1, 2, 10, 实现了先入后出, 在我们手动压栈后, 栈显示非空, 在我们弹完所有的元素后, 栈显示为空。结果完美符合我们的构想。

2. 队列的实现

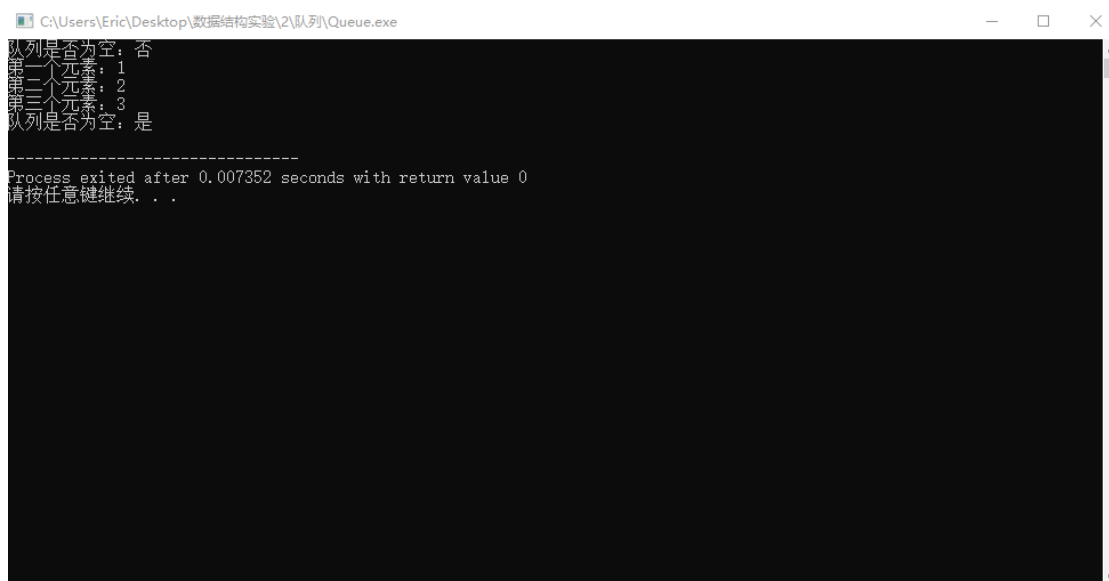


图 2 运行结果截图

在我们输入 1, 2, 3 后, 其出队列输出也为 1, 2, 3, 即实现了先入先出的结构, 在我们手动入队列后, 队列显示非空, 在我们手动对所有的元素出队列后, 队列显示为空, 符合我们的预料及队列的全部特性。

【实验心得】

考虑到在实验 1, 我们已经完美实现 ADT 表的基本操作, 于是我就想到, 我们完全可以通过我们已经构建的函数来实现这两个数据结构。

在本次实验, 我深深地感受到了整体项目构建带来的力量, 在上次成功的 ADT 表构建下, 本次实验即为在固定位置对于 ADT 表的插入, 查表与删除, 均可以通过调用 ADT 的函数来实现, 避免

了重复造轮子的痛苦。整体代码写得比较简洁了。

本次实验也存在一些问题，和一些可改进的方面：

入队列效率问题的改进：

在本次实验中，我们直接调用了上次实验的函数来完成实验，但这可能对于实现队列的入队操作并没有做到最佳效率，原函数是遍历至队尾然后插入数据：

```
Node* current = list->head;
for (int pos = 1; pos < i; pos++) {
    current = current->next;
}
```

很明显，这样对于队尾元素的插入的效率是 $O(n)$ ，但是一般情况下，我们希望入队列的效率为 $O(1)$ ，这说明这样的操作是有点弄巧成拙了，为改善这个问题，我们可以改进链表的定义结构，定义一个尾指针：

```
typedef struct {
    Node* head;
    int size;
    Node* tail;
} List;
```

后续在虽然在插入元素的复杂度会高一点，但是对于入队操作，我们直接让数据插入尾指针指向的位置，这将大幅提高队列的效率。